1.2 Erarbeitungsphase

Datenbank für eine Buchtausch App

Genre

Entität:

```
create table if not exists Genre(
   Id bigint unsigned not null primary key auto_increment,
   Name varchar(100) not null
);
```

Testabfrage:

```
select Name from genre;
```

Ergebnis:

Name
Fantasy
Science Fiction
Romance
Thriller
Horror
Drama
Biography
History
Programming
Philosophy

Author

Entität:

```
create table if not exists Author(
   Id bigint unsigned not null primary key auto_increment,
   FirstName varchar(100) not null,
   LastName varchar(100) not null
);
```

Testabfrage:

```
select FirstName, LastName from author
where FirstName like '%J%';
```

Ergebnis:

FirstName	LastName
J.K.	Rowling
Jane	Austen
J.R.R.	Tolkien

Language

Entität:

```
create table if not exists Language(
   Code varchar(3) not null primary key,
   Name varchar(100) not null
);
```

Testabfrage:

```
select Name from language
where Code = 'DE'
```

Ergebnis:

Name
Deutsch

Publisher

Entität:

```
create table if not exists Publisher(
   Id bigint unsigned not null primary key auto_increment,
   Name varchar(255) not null
);
```

Testabfrage:

```
select Name from Publisher
where Id between 2 and 4;
```

Ergebnis:

Name
HarperCollins
Springer
OReilly

Address

Entität:

```
create table if not exists Address(
   Id bigint unsigned not null primary key auto_increment,
   Street varchar(255) not null,
   Postcode char(5) not null,
   City varchar(100) not null,
   Country varchar(100) not null
);
```

Testabfrage:

```
select City from address
where country = 'Germany’;
```

Ergebnis:

City
Berlin
München
Hamburg
Köln
Frankfurt
Stuttgart
Düsseldorf
Bremen
Dresden
Nürnberg

Location

Entität:

```
create table if not exists Location(
   Id bigint unsigned not null primary key auto_increment,
   Latitude decimal(9,6) not null,
   Longitude decimal(9,6) not null,
   Address_Id bigint unsigned not null,

   foreign key(Address_Id) references Address(Id)
   on delete cascade
   on update cascade
);
```

Testabfrage:

```
select a.Street from Location l
join Address a on l.Address_Id = a.Id
where l.Longitude > 10;
```

Ergebnis:

Street
Hauptstr. 1
Bahnhofstr. 2
Seestr. 9
Wiesenweg 10

Book

Entität:

```
create table if not exists Book(
   Id bigint unsigned not null primary key auto_increment,
   Isbn char(13) not null unique,
   Title varchar(255) not null,
   ReleaseDate date not null,
   Publisher_Id bigint unsigned not null,
   Language_Code varchar(3) not null,

   foreign key(Publisher_Id) references Publisher(Id) on delete cascade on update cascade,
   foreign key(Language_Code) references Language(Code) on delete cascade on update cascade
);
```

Testabfrage:

```
select Title from book where year(ReleaseDate) < 2000;
```

Ergebnis:

Title
Harry Potter
1984
The Shining
Pride and Prejudice
The Hobbit
Foundation
Also sprach Zarathustra

User

Entität:

```
create table if not exists User(
   Id bigint unsigned not null primary key auto_increment,
   Email varchar(255) not null unique,
   FirstName varchar(100) not null,
   LastName varchar(100) not null,
   PhoneNumber varchar(50) not null,
   JoinedDate date not null
);
```

Testabfrage:

```
select FirstName from user
where day(JoinedDate) > 10;
```

Ergebnis:

FirstName
Max
Lisa
Sara
Paul
Emma
Noah
Mia

Bookcopy

Entität:

```
create table if not exists Bookcopy(
   Id bigint unsigned not null primary key auto_increment,
   Book_Id bigint unsigned not null,
   User_Id bigint unsigned not null,
   Location_Id bigint unsigned not null,
   `Condition` varchar(50) not null,

   foreign key(Book_Id) references Book(Id) on delete cascade on update cascade,
   foreign key(User_Id) references User(Id) on delete cascade on update cascade,
   foreign key(Location_Id) references Location(Id) on delete cascade on update cascade
);
```

Testabfrage:

```
select b.Title from bookcopy c
join book b on b.Id = c.Book_Id where `Condition` = 'Sehr Gut';
```

Ergebnis:

Title
Harry Potter
Pride and Prejudice
The Hobbit
Also sprach Zarathustra

DeliveryOption

Entität:

```
create table if not exists DeliveryOption(
   Id bigint unsigned not null primary key auto_increment,
   Bookcopy_Id bigint unsigned not null,
   IsDeliveryPossible boolean not null,

   foreign key(Bookcopy_Id) references Bookcopy(Id)
   on delete cascade
   on update cascade
);
```

Testabfrage:

```
select b.Title from deliveryoption d
join bookcopy c on d.Bookcopy_Id = c.Id
join book b on c.Book_Id = b.Id where d.IsDeliveryPossible;
```

Ergebnis:

Title
Harry Potter
The Shining
Pride and Prejudice
Clean Code
Da Vinci Code
Foundation

Availability

Entität:

```
create table if not exists Availability(
   Id bigint unsigned not null primary key auto_increment,
   Bookcopy_Id bigint unsigned not null,
   MaxDays int unsigned not null,

   foreign key(Bookcopy_Id) references Bookcopy(Id) on delete cascade on update cascade
);
```

Testabfrage:

```
select Id from availability where MaxDays = 14;
```

Ergebnis:

Id
1
4
7
10

Lending

Entität:

```
create table if not exists Lending(
   Id bigint unsigned not null primary key auto_increment,
   Bookcopy_Id bigint unsigned not null,
   User_Id bigint unsigned not null,
   StartDate date not null,
   EndDate date not null,

   foreign key(Bookcopy_Id) references Bookcopy(Id) on delete cascade on update cascade,
   foreign key(User_Id) references User(Id) on delete cascade on update cascade,

   check(EndDate >= StartDate)
);
```

Testabfrage:

```
select Id from lending where User_Id = 2;
```

Ergebnis:

Id
1
11

Rating

Entität:

```
create table if not exists Rating(
   Id bigint unsigned not null primary key auto_increment,
   Book_Id bigint unsigned not null,
   User_Id bigint unsigned not null,
   Stars tinyint unsigned not null,
   Comment text not null,

   foreign key(Book_Id) references Book(Id) on delete cascade on update cascade,
   foreign key(User_Id) references User(Id) on delete cascade on update cascade,

   check(Stars between 1 and 5)
);
```

Testabfrage:

```
select Comment from Rating where Stars <= 3;
```

Ergebnis:

Comment
Unterhaltsam

BookAuthor

Entität:

```
create table if not exists BookAuthor(
   Id bigint unsigned not null primary key auto_increment,
   Book_Id bigint unsigned not null,
   Author_Id bigint unsigned not null,

   foreign key(Book_Id) references Book(Id) on delete cascade on update cascade,
   foreign key(Author_Id) references Author(Id) on delete cascade on update cascade
);
```

Testabfrage:

```
select b.Title as BookTitle, a.FirstName as AuthorFirstName, a.LastName as AuthorLastName
from BookAuthor ba
join Book b on ba.Book_Id = b.Id
join Author a on ba.Author_Id = a.Id
order by b.Title;
```

Ergebnis:

BookTitle	AuthorFirstName	AuthorLastName
1984	George	Orwell
Also sprach Zarathustra	Friedrich	Nietzsche
Clean Code	Robert	Martin
Da Vinci Code	Dan	Brown
Foundation	Isaac	Asimov
Harry Potter	J.K.	Rowling
Pride and Prejudice	Jane	Austen
Sapiens	Yuval Noah	Harari
The Hobbit	J.R.R.	Tolkien
The Shining	Stephen	King

BookGenre

Entität:

```
create table if not exists BookGenre(
   Id bigint unsigned not null primary key auto_increment,
   Book_Id bigint unsigned not null,
   Genre_Id bigint unsigned not null,

   foreign key(Book_Id) references Book(Id) on delete cascade on update cascade,
   foreign key(Genre_Id) references Genre(Id) on delete cascade on update cascade
);
```

Testabfrage:

```
select b.Title as BookTitle, g.Name as Genre
from BookGenre bg
join Book b on bg.Book_Id = b.Id
join Genre g on bg.Genre_Id = g.Id
order by b.Title;
```

Ergebnis:

BookTitle	Genre
1984	Science Fiction
Also sprach Zarathustra	Philosophy
Clean Code	Programming
Da Vinci Code	Thriller
Foundation	Science Fiction
Harry Potter	Fantasy
Pride and Prejudice	Romance
Sapiens	History
The Hobbit	Fantasy
The Shining	Horror

Zusammenfassung der Implementierung

Die Datenbank für die Buchtausch-App wurde mit MySQL auf Basis eines relationalen Datenbankmodells umgesetzt.

Zunächst wurde ein ER-Modell erstellt, aus dem anschließend die Tabellen mit SQL CREATE TABLE Statements abgeleitet wurden.

Jede Entität besitzt einen Primärschlüssel, um Datensätze eindeutig zu identifizieren.

Die Beziehungen zwischen den Tabellen wurden über Fremdschlüssel realisiert.

Dadurch wird die Datenintegrität sichergestellt und verhindert, dass ungültige Referenzen entstehen.

Für viele-zu-viele-Beziehungen wurden zusätzliche Beziehungstabellen wie BookAuthor und BookGenre erstellt.

Zur Überprüfung der Implementierung wurden Testdaten eingefügt und für jede Entität Beispielabfragen mit SELECT erstellt.

Die Ergebnisse dieser Abfragen wurden im Datenbankmanagementsystem geprüft und in der Präsentation dokumentiert.